CODE REVIEW

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Prepare Small, Well-Described Changes

- Keep each PR focused on one logical unit of work before submitting for AI review.
- Write a clear description explaining what changed, why, and how it was tested.
- Your description now serves two audiences: human teammates and the LLM reviewer, which uses it to seed its understanding of the change.

Reasoning: Why Small PRs Review Better

- Review at 200 LOC/hour found ~2/3 of design defects and >1/2 of code defects
- Median review sizes converged to small across Microsoft, Google, AMD, and open-source
- More files in a change = lower proportion of valuable review comments
- Small change lists and clear descriptions make review scale at Google
- LLM reviewer precision falls below 10% on large diffs due to false positives

Key Practices: Keeping the Change Small

- One logical change per PR: separate refactoring, renaming, and feature additions into different PRs
- If you cannot read the diff in 30 minutes, it is too big; split it into a stack of smaller PRs
- Ask the AI: "list the logical chunks in this diff and suggest a commit order for stacked PRs"

Key Practices: Writing the Description

- Include four things: what changed, why, how you tested it, and what you want the reviewer to check
- Name invariants the change must preserve (e.g., public API compatibility, thread safety, no extra DB calls)
- State what you considered and rejected as this primes both human and AI reviewers to focus on design decisions
- Do not rely on the diff alone as "The commits explain it" is not a description

Guideline 2: Feed the Reviewer the Surrounding Code

- Do not send only the changed lines when running the AI review.
- Include the enclosing function or class, relevant imports, type definitions, and at least one call site for public functions.
- The PR description gives the AI intent; the surrounding code gives the AI the mechanics.

Reasoning: Why Context Matters

- Insufficient context causes frequent false positives and trust issues in AI-assisted review
- Simple prompts without the enclosing function "do not work well in practice"
- Context-extended diffs using AST boundaries yield more reliable review comments

Key Practices: Preparing the Prompt

- Paste the full enclosing function or class, not just the changed lines
- Mark changed lines clearly (e.g., with + and -)
- Include relevant imports and any type definitions used in the changed code
- If the change touches a public function, include at least one call site so the model can reason about callers

Key Practices: Checking Whether Context Was Enough

- Scan AI comments for references to code you did not provide as these are guesses, not findings
- If the AI asks for more information, answer it and re-run the review
- Re-run after adding missing context; compare both passes and keep only comments that appear in both or have clear evidence
- If context is a setting in your tool, then include enough context surrounding the changes

Guideline 3: Run the Review as Two Passes

- Structure the AI review as two prompts, not one.
- Pass 1 (comprehension): ask the AI to explain what the change does in its own words and what invariants it should preserve.
- Pass 2 (rubric): ask the AI to evaluate the change against an explicit rubric covering security, performance, style etc and what to skip.

Reasoning: Why Two Passes Work Better

- Code understanding is the key aspect of code reviewing as defect detection is downstream of comprehension
- If the AI's summary of the change is wrong, every critique comment inherits the same misunderstanding
- Shorter, structured rubrics produce better AI output than long narrative prose

Key Practices: The Comprehension Pass

- Ask the AI to describe the change in 3 to 5 sentences without using the words "bug" or "issue"
- Ask it to state the intent it infers and what invariants the change should preserve
- Read the summary before looking at any critique and if the summary is wrong, fix the prompt before running the rubric pass

Key Practices: The Rubric Pass

- Use short imperative rules, one per line, under clear headings (Security, Performance, Error handling, Style)
- Scope rules to the project and PR like "flag SQL injection" is more useful than "review for quality"
- Include negative rules: things you want the AI to skip (e.g., "do not comment on line length, a linter handles that")
- Keep the rubric short as shorter files are followed more reliably
- Store the rubric in version control so the whole team can evolve it

References [1-5]

- [1] Alami et al. (2025). Accountability in Code Review: The Role of Intrinsic Drivers and the Impact of LLMs. ACM TOSEM. DOI: 10.1145/3721127.
- [2] Kawalerowicz et al. (2026). LLMs as Code Review Agents: A Rapid Review and Experimental Evaluation. Springer LNNS. DOI: 10.1007/978-3-032-09318-9_24.
- [3] Adalsteinsson et al. (2025). Rethinking Code Review Workflows with LLM Assistance: An Empirical Study. arXiv:2505.16339.
- [4] Ramesh et al. (2025). Automated Code Review Using LLMs at Ericsson: An Experience Report. ICSME 2025. arXiv:2507.19115.
- [5] Haider et al. (2024). Prompting and Fine-tuning LLMs for Automated Code Review Comment Generation. arXiv:2411.10129.

References [6-10]

- [6] Zhang et al. (2025). LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM. ASE 2025. arXiv:2512.01356.
- [7] GitHub Engineering. (2025). Code review in the age of AI: Why developers will always own the merge button. The GitHub Blog.
- [8] Uber Engineering. (2025). uReview: Scalable, Trustworthy GenAI for Code Review at Uber. Uber Blog.
- [9] Romeo and Conti. (2025). Exploring automation bias in human-AI collaboration. AI & Society. DOI: 10.1007/s00146-025-02422-7.
- [10] GitHub Docs. (2026). Using custom instructions to unlock the power of Copilot code review. docs.github.com.

References [11-15]

- [11] GitHub Engineering. (2025). Unlocking the full power of Copilot code review. The GitHub Blog.
- [12] Cihan et al. (2025). Evaluating Large Language Models for Code Review. arXiv:2505.20206.
- [13] Zeng et al. (2025). Benchmarking and Studying the LLM-based Code Review. arXiv:2509.01494.
- [14] CodeRabbit. (2025). State of AI vs Human Code Generation Report. coderabbit.ai.
- [15] Cotroneo et al. (2025). Human-Written vs. AI-Generated Code: A Large-Scale Study. arXiv:2508.21634.

References [16-20]

- [16] Apiiro. (2025). 4x Velocity, 10x Vulnerabilities: AI Coding Assistants Are Shipping More Risks. Apiiro Blog.
- [17] Liu et al. (2024). Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. arXiv:2404.00971.
- [18] Lamb et al. (2025). Detecting malicious pull requests at scale with LLMs. Datadog Engineering Blog.
- [19] Bacchelli and Bird. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. ICSE. DOI: 10.1109/ICSE.2013.6606617.
- [20] Rigby and Bird. (2013). Convergent Contemporary Software Peer Review Practices. ESEC/FSE. DOI: 10.1145/2491411.2491444.

References [21-25]

- [21] Kemerer and Paulk. (2009). The Impact of Design and Code Reviews on Software Quality. IEEE TSE. DOI: 10.1109/TSE.2009.27.
- [22] Sadowski et al. (2018). Modern Code Review: A Case Study at Google. ICSE-SEIP. DOI: 10.1145/3183519.3183525.
- [23] Bosu et al. (2015). Characteristics of Useful Code Reviews: An Empirical Study at Microsoft. MSR. DOI: 10.1109/MSR.2015.21.
- [24] Spadini et al. (2020). Primers or Reminders? The Effects of Existing Review Comments on Code Review. ICSE. DOI: 10.1145/3377811.3380385.
- [25] Heander et al. (2025). Code Review as Decision-Making: Building a Cognitive Model from the Questions Asked During Code Review. arXiv:2507.09637.